Koen Essers • Master Internship • April 12, 2026

Notes Week 11

a. New Wind Velocity

I updated the wind velocity in both the *wind angular momentum loss* prescription, as well as in the *wind mass loss* prescription. Both prescriptions now use a constant wind velocity of 15 km s^{-1}.

b. Updated Reference Single Star with Updated Wind and History Data

b.1 Updated Wind and History Data

The single star now saves `I_eff`, `rg2`, `M_conv` and `R_conv` to give `evolve.py` everything it needs to accurately compute the orbital evolution.

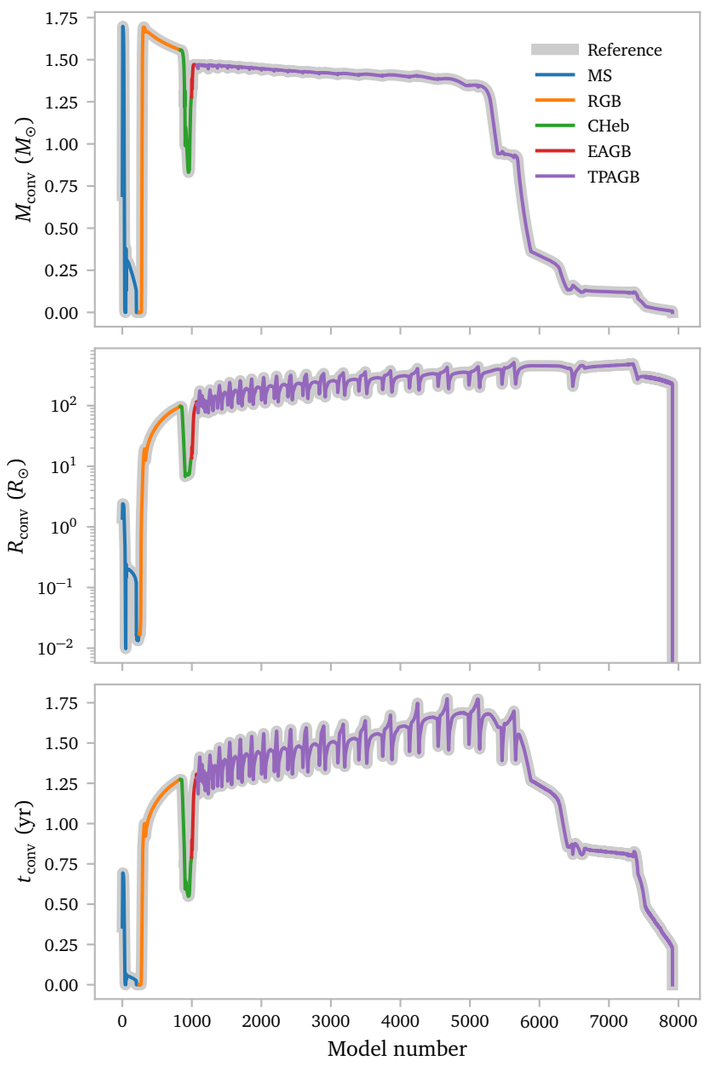
b.2 Updated Model saving procedure

c. Comparing with evolve.py

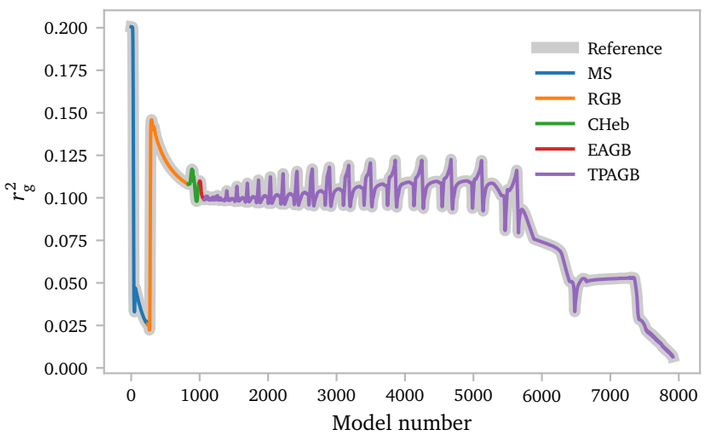
Before, I only loaded the TPAGB data from the Rees models into the Star class from `evolve.py`. To accurately predict starting Roche Lobe radii, q-values and stellar spin parameters, it is important to model the previous evolutionary phases as well. To do this, I modify the `read_stellar_models` function to read the complete standard star data:

```
def read_stellar_models():
    n = [0]
    for phase in ["MS", "GB", "CHeB", "EAGB", "TPAGB"]:
        data = mr.MesaData(
            f"""/home/koen/master-internship/mesa-models/
            standard-2msun-v2/LOGS/{phase}/history.data"""
        )
        n.append(n[-1] + len(data.model_number[1:]))

    combined_star = {}
    for bulk_name in data.bulk_names:
        combined_star[bulk_name] = np.empty(n[-1])
    combined_star["phase"] = np.empty(n[-1])

    for i, phase in enumerate(["MS", "GB", "CHeB", "EAGB", "TPAGB"]):
        data = mr.MesaData(
            f"""/home/koen/master-internship/mesa-models/
            standard-2msun-v2/LOGS/{phase}/history.data"""
        )
        for bulk_name in data.bulk_names:
            if bulk_name in ["model_number", "star_age"] and i != 0:
                combined_star[bulk_name][n[i] : n[i + 1]] = (
                    data.data(bulk_name)[1:] + combined_star[bulk_name][n[i] - 1]
                )
            else:
                combined_star[bulk_name][n[i] : n[i + 1]] = data.data(bulk_name)[1:]
        combined_star["phase"][n[i] : n[i + 1]] = i * np.ones(n[i + 1] - n[i])

    Stars = [StellarModel(combined_star, n)]
    return Stars
```

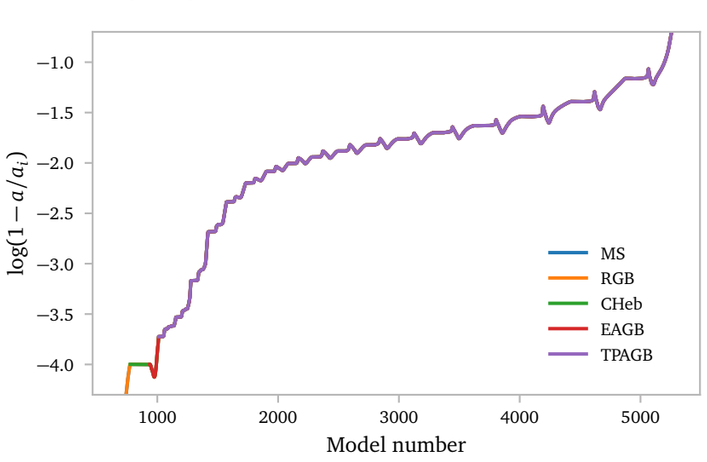
Below, I show that this ensures the `Star` class from `evolve.py` has the same `M_conv`, `R_conv`, `t_conv` and `rg2` values as the MESA reference star. It also has access to all other properties required for computing the tides and wind.



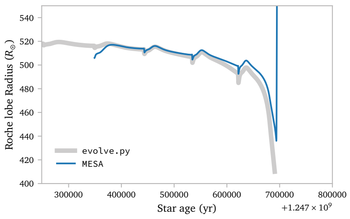
Now for r_g^2



If we use these values to compute the wind and tides, we can see the effects of all phases on the separation and Roche lobe radius for arbitrary staring radii and mass ratios.



Now, we can compare the Roche lobe radius from `evolve.py` with the radius from `MESA`. If everything is correct, it *should* be nearly identical.



Clearly, it is *not* nearly identical. The `evolve.py` model seems to show a much sharper decline in Roche lobe radius. My suspicion was that this is due to the stellar wind prescription, since it does not seem to affect the first and second pulse as much as it affects the third and final pulse.

Below, I show the code that I use that is responsible for the angular momentum loss due to winds:

```
! src/bin/additional_routines.inc
subroutine wind_loss(binary_id, ierr)
    integer, intent(in) :: binary_id
    integer, intent(out) :: ierr
    type(binary_info), pointer :: b
    real(dp) :: q, vw_over_vorb
    real(dp) :: beta, eta
    real(dp) :: omega, I_eff
    real(dp) :: domega_dt_wind

    ierr = 0
    call binary_ptr(binary_id, b, ierr)
    if (ierr /= 0) then
        write (*, *) 'failed in binary_ptr'
        return
    end if

    ! --- compute q == m_accretor / m_donor and v
    q = b%m(b%a_i)/b%m(b%d_i)
    vw_over_vorb = compute_vwind(b%s_donor)/compute_vorbit(b%m(1) + b%m(2), b%separation)
    b%s_donor%xtra(x_vw_over_vorb) = vw_over_vorb

    ! --- get beta and eta from Saladino
    beta = compute_beta_Sal(q, vw_over_vorb)/sqrt(1 - pow2(b%eccentricity))
    b%s_donor%xtra(x_beta) = beta
    eta = compute_eta_Sal(q, vw_over_vorb)
    b%s_donor%xtra(x_eta) = eta

    ! --- star angular momentum loss due to winds
    ! compute the change in spin frequency. this
    ! gets used in extra_binary_finish_step.
    omega = b%s_donor%xtra(x_omega)
    I_eff = b%s_donor%xtra(x_I_eff)
    domega_dt_wind = (b%mdot_system_wind(b%d_i) &
                     *(b%s_donor%photosphere_r*Rsun)**2*omega/1.5)/I_eff
    b%s_donor%xtra(x_domega_dt_wind) = domega_dt_wind

    ! --- mass lost from vicinity of donor
    b%jdot_ml = b%mdot_system_wind(b%d_i) &
                *(1 - beta) &
                *eta &
                *(b%separation)**2 &
                *sqrt(1 - pow2(b%eccentricity)) &
                *2*pi/b%period

end subroutine wind_loss
```

The important flaw here is the use of the variable `b%mdot_system_wind(b%d_i)` in combination with the term `(1-beta)`.

If we look at how `b%mdot_system_wind(b%d_i)` is defined in the MESA source code:

```
b% mdot_system_wind(b% d_i) = b% s_donor% mstar_dot - b% mtransfer_rate &
            + b% mdot_wind_transfer(b% a_i) - b% mdot_wind_transfer(b% d_i)
```

we can see that it uses the total $\dot{M}$ for the donor star, subtracts the mass transfer term due to RLOF `b% mtransfer_rate`, adds the wind mass transfer from the accretor `b% mdot_wind_transfer(b% a_i)`, which is always zero if the accretor is a point mass, and subtracts the wind mass transfer from the donor `b% mdot_wind_transfer(b% d_i)`. Effectively, this means

$$\dot{M}_{\rm wind,sys} = \dot{M}_{\rm d,wind} - \dot{M}_{\rm d,wind\text{-}transfer},$$

and the problem is already becoming clear. Below, I show the definition of `b% mdot_wind_transfer(b% d_i)`:

```
! $MESA_DIR/binary/private/binary_mdot.f90
! solve wind mass transfer
    ! b% mdot_wind_transfer(b% d_i) is a negative number that gives the
    ! amount of mass transferred by unit time from the donor to the
    ! accretor.
    call eval_wind_xfer_fractions(b% binary_id, ierr)
    if (ierr/=0) then
        write(*,*) "Error in eval_wind_xfer_fractions"
        return
    end if
    b% mdot_wind_transfer(b% d_i) = b% s_donor% mstar_dot * &
        b% wind_xfer_fraction(b% d_i)
```

Here, `b% wind_xfer_fraction(b% d_i)` is exactly `beta`, so really,

$$\dot{M}_{\rm wind,sys} = \dot{M}_{\rm d,wind}(1-\beta),$$

and my subroutine above is effectively computing

$$\dot{J} = \dot{M}_{\rm wind}(1-\beta)^2\eta a^2\Omega_{\rm orb}.$$

which is *wrong*! To check whether this truly is the cause of the discrepancy, I decide to modify the `evolve.py` script to also use and extra factor of $1-\beta$ in the $\dot{a}/a$ computation, since this is *much* quicker than simulating the possibly correct result in MESA. To see how the extra factor of $1-\beta$ affects the separation, I derived the following expression using Saladino et al. (2018):

$$\begin{aligned}\left(\frac{\dot{a}}{a}\right)_{\rm wind} &= 2\left(\frac{\dot{J}}{J}\right)_{\rm wind} - 2\frac{\dot{M}_{1,\rm wind}}{M_1} - 2\frac{\dot{M}_{2,\rm wind}}{M_2} + \frac{\dot{M}_{1,\rm wind}-\dot{M}_{2,\rm wind}}{M_1+M_2}\\ &= 2\left(\frac{\dot{J}}{J}\right)_{\rm wind} - 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}} + 2\frac{\beta\dot{M}_{\rm d,wind}}{M_{\rm a}} + \frac{\dot{M}_{\rm d,wind}-\beta\dot{M}_{\rm d,wind}}{M_{\rm d}+M_{\rm a}}\end{aligned}$$

From the expression above, and $J=\mu a^2\Omega_{\rm orb}$, we can deduce that

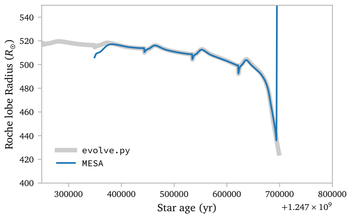
$$\begin{aligned}\frac{\dot{J}}{J} &= \frac{\dot{M}_{\rm d,wind}(1-\beta)^2\eta(M_{\rm d}+M_{\rm a})}{M_{\rm d}M_{\rm a}} \quad \text{and thus}\\ \left(\frac{\dot{a}}{a}\right)_{\rm wind} &= 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}}\left[\frac{(1-\beta)^2\eta(M_{\rm d}+M_{\rm a})}{M_{\rm a}} - 1 + \frac{\beta M_{\rm d}}{M_{\rm a}} + \frac{M_{\rm d}(1-\beta)}{2(M_{\rm d}+M_{\rm a})}\right]\end{aligned}$$

With $\tilde{q}=M_{\rm d}/M_{\rm a}$:

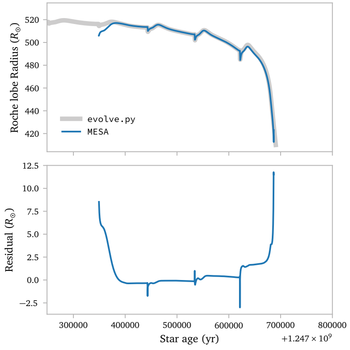
$$\frac{\dot{J}}{J} = 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}}\left[(1-\beta)^2\eta(1+\tilde{q}) - 1 + \tilde{q}\beta + \frac{\tilde{q}(1-\beta)}{2(1+\tilde{q})}\right].$$

This expression differs only in the in the $(1-\beta)^2$ term. The correct equation only has $(1-\beta)$.

If we use this expression in `evolve.py`, we see the following:

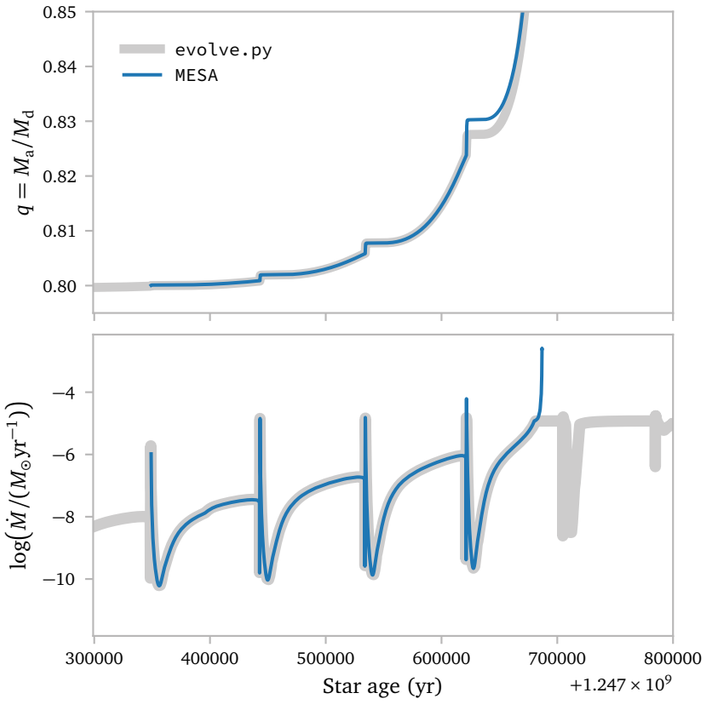


Excellent agreement! This makes me optimistic that the accidental double $(1-\beta)$ factor is the only (significant) discrepancy between the two codes. Next, I fix the mistake and run a new MESA model.



The `evolve.py` and MESA implementation agree very well *but* after the second pulse, the `evolve.py` implementation seems to predict slightly higher Roche lobe radii. This is more apparent in the bottom panel, where I plot the residuals.

I predict this is due to non-negligible amounts of RLOF mass transfer during the thermal pulses. To confirm this, we can plot M for both methods, as well as q for both methods. This can also explain the slight variations in TPAGB interpulse durations.



These plots clearly show that RLOF affects the mass ratio, and thus also the Roche lobe radius.

d. Grid

e. Analytic Solutions

e.1 Predicting Beta Values

References

Saladino, M. I., Pols, O. R., van der Helm, E., Pelupessy, I., & Portegies Zwart, S. (2018) *Gone with the wind: the impact of wind mass transfer on the orbital evolution of AGB binary systems*. A&A618, A50.